# Laboratorio 4 - Polimorfismo

**ArenaGT | CC2008 - Programación Orientada a Objetos**

## 1. Requisitos funcionales

RF01. Registrar participantes con código único, nombre y empresa; validar texto no vacío.

RF02. Crear torneos con código único, nombre y disciplina; mantener estado INSCRIPCION, EN_JUEGO o FINALIZADO.

RF03. Cotizar inscripción por disciplina y cantidad sin cobrar ni modificar registros.

RF04. Inscribir únicamente durante INSCRIPCION; aplicar máximo de cupos, prohibición de duplicados y máximo dos representantes de una empresa.

RF05. Cerrar inscripción solo al alcanzar el mínimo y generar automáticamente todas las parejas o una carrera de relevos.

RF06. Programar y reprogramar eventos no finalizados con fecha, hora de 0 a 23 y sede, impidiendo conflictos de sede y participantes globalmente.

RF07. Registrar un resultado válido únicamente una vez en un evento programado; actualizar estadísticas de forma atómica.

RF08. Ordenar posiciones por puntos, diferencia, victorias y nombre; determinar campeón y premio automáticamente al terminar todos los eventos.

RF09. Mostrar calendario general, por torneo y por participante, en fecha/hora/sede ascendente.

RF10. Reportar ingresos por disciplina y totales, premios pagados, clasificaciones, campeones y ranking de empresas.

RF11. Continuar el menú hasta salir, mostrando errores de entradas sin terminar la aplicación.

RF12. Precargar participantes y torneos de demostración sin registrar cobros por esos inscritos.

## 2. Reglas de negocio por disciplina

| Disciplina | Cupo | Cuota | Criterio de puntos y diferencia |
|---|---|---|---|
| Fútbol sala | 4 a 8 | Q600 | Victoria 3, empate 1, derrota 0; diferencia de goles. |
| Fútbol sala con penales | 4 a 8 | Q750 | Si goles distintos: 3/0; con goles iguales: penales obligatorios y distintos, 2/1; diferencia solo goles. |
| Baloncesto | 4 a 6 | Q500 | No hay empates, victoria 2, derrota 1; diferencia de puntos. |
| Voleibol | 4 a 6 | Q450 | Ganador con 3 sets; 3-0/3-1 da 3/0; 3-2 da 2/1; diferencia de sets. |
| Relevos | 3 a 8 | Q300 | Una carrera; tiempos únicos positivos o DQ; 10,8,6,5,4,3,2,1; DQ=0; diferencia=0. |

## 3. Datos conservados, operaciones y validación

**Datos:** participantes por código; torneos y su disciplina; inscripciones y tabla acumulada; eventos, calendario, resultados ya aplicados (mediante marca de finalización y estadística acumulada); ingresos cobrados y premio liquidado por torneo. No se persiste información en archivos.

**Operaciones:** alta, creación, cotización, inscripción, cierre, programación, reprogramación, resultado, consultas y reportes. Todas validan antes de modificar el estado. El libro de cobros es independiente de las inscripciones precargadas.

**Invariantes:** códigos no vacíos/únicos, cuotas positivas, cupos mínimos/máximos, límite por empresa, eventos sin doble resultado, hora válida, no duplicidad espacial o temporal por participante, marcadores y tiempos válidos, premio solo si todos los eventos se completan.

## 4. Estados y transiciones

| Objeto | Origen | Operación y condición | Destino |
|---|---|---|---|
| Torneo | INSCRIPCION | cerrar(), si inscritos ≥ mínimo | EN_JUEGO |
| Torneo | EN_JUEGO | último evento válido finalizado | FINALIZADO |
| Evento | SIN PROGRAMAR | programar() sin conflictos | PROGRAMADO |
| Evento | PROGRAMADO | programar() sin conflictos | PROGRAMADO (reprogramado) |
| Evento | PROGRAMADO | registrar() válido | FINALIZADO |

## 5. Abstracciones y colaboración entre objetos

**Disciplina** (abstracta) encapsula nombre, cupos, cuota, cotización y contrato abstracto crearEventos/evaluar. **DisciplinaEncuentros** (abstracta) implementa la generación de combinaciones únicas; **FutbolSala**, **FutbolPenales**, **Baloncesto** y **Voleibol** sobrescriben reglas de puntuación. **FutbolPenales hereda de FutbolSala** y modifica únicamente cuota y definición de resultados. **Relevos** genera una Carrera y evalúa los tiempos.

**Evento** (abstracta) comparte código, torneo, programación, finalización y orden mediante Comparable; **Encuentro** y **Carrera** resuelven participantes concretos. **Resultado** es una interfaz de datos implementada por Marcador, ConPenales y RelevoResultado. **Calendario** implementa Iterable y centraliza filtrado y orden. **Torneo.Estadistica** implementa Comparable para desempates.

**Torneo** conserva inscripciones, eventos, tabla, estado y premio; **Sistema** orquesta los registros y conflictos entre torneos; **Main** tiene main(String[]) y gestiona exclusivamente la interacción de consola. La evaluación de puntajes no está en Main ni en Sistema.

**Inicialización:** los constructores reciben atributos obligatorios y verifican datos. Los constructores concretos de disciplinas invocan super(...) para inicializar cupos/cuotas; cada Evento concreto invoca super(torneo) para crear su identidad y relación. Los mapas y listas se crean vacíos y solo se exponen en vistas no modificables.

## 6. Justificación y extensibilidad

Los encuentros y las carreras comparten planificación, cronología, estado y pertenencia a torneo, modelados con Evento abstracta; sus listas de participantes se resuelven en subclases. Así no se duplican controles de calendario. Fútbol sala con penales es una especialización de fútbol sala: reutiliza estructura y modifica la regla de desempate/puntuación. Una sola clase con banderas o switch por tipo mezclaría reglas y exigiría cambios en varios puntos.

Para sumar una disciplina se extiende Disciplina o DisciplinaEncuentros y se implementan crearEventos/evaluar; se registra su instancia en el catálogo de Sistema. Para un evento nuevo se extiende Evento y se proporciona la lista de participantes. Calendario, tablas y ranking no necesitan modificar su algoritmo. El menú actual requiere adaptar únicamente la captura de un nuevo formato de Resultado; la lógica de puntuación permanece polimórfica.

La composición de Torneo a Evento representa su ciclo de vida dependiente; Sistema administra los registros de Torneo y Participante; Calendario solo se asocia con Evento (referencias temporales, no propiedad). Torneo se asocia con Participante y Disciplina. Las multiplicidades y tipos se representan en el diagrama de la sección siguiente.

## 7. Diagrama UML de clases

Notación: triángulo hueco = herencia; línea discontinua con triángulo = realización; rombo negro = composición; línea sin rombo = asociación. Las clases abstractas, contratos y métodos sobrescritos están identificados.

Los diagramas UML completos en formato vectorial se incorporan como **tres páginas A3 al final de este mismo PDF**, para permitir lectura y ampliación. El primero documenta gestión y torneos, el segundo los eventos y calendarios, y el tercero las disciplinas y la interfaz Resultado. Los tres se generaron a partir de las firmas compiladas de las clases Java, identificando la visibilidad y los tipos.

Los métodos indican sus tipos, parámetros y visibilidad. Los contratos Comparable<Evento>, Comparable<Estadistica> e Iterable<Evento> aparecen mediante relaciones de realización.

## 7.1. Contratos y decisiones específicas

Los métodos **Disciplina.leerResultado(Scanner, Evento)**, **Disciplina.crearEventos(Torneo)** y **Disciplina.evaluar(Evento, Resultado)** son contratos polimórficos. Main invoca leerResultado sobre Disciplina sin conocer la clase concreta. El despacho virtual de Evento.comoEncuentro()/comoCarrera() y Resultado.comoMarcador()/comoPenales()/comoRelevo() valida compatibilidad mediante métodos sobrescritos, sin instanceof ni conversiones explícitas para decisiones por disciplina. No se calcula la puntuación mediante condicionales de tipo en el controlador.

## 8. Casos de prueba ejecutados

Las siguientes verificaciones se ejecutaron mediante **java -cp bin arenagt.Pruebas**. “OK” significa que el resultado obtenido coincidió con el esperado. Se incluyen entradas válidas, inválidas, cambios de estado y preservación de ingresos.

## 8.1. Matriz de entradas, esperados y obtenidos

| Entrada concreta | Resultado esperado | Obtenido |
|---|---|---|
| Fútbol sala 4-1 | 3/0 pts; diferencia +3/-3 | 3/0; +3/-3 |
| Fútbol sala 2-2 | 1 punto para cada uno | 1/1 |
| Penales: goles 1-1, penales 4-3 | 2/1 pts; diferencia 0; 1 victoria | 2/1; 0; 1 |
| Penales: goles 0-0, penales 2-2 | Rechazar sin cambiar tabla | Rechazado; tabla intacta |
| Baloncesto 70-70 | Rechazar y no acumular puntos | Rechazado; 0 acumulado |
| Baloncesto 70-60 | 2/1 pts; diferencia +10/-10 | 2/1; +10/-10 |
| Voleibol 3-0, 3-1, 3-2 | 3/0, 3/0, 2/1 pts | 3/0, 3/0, 2/1 |
| Relevos: 50, 51, 52, DQ | 10, 8, 6, 0; diferencia 0 | 10, 8, 6, 0; 0 |
| Relevos: 50, 50, 52 | Rechazar tiempos duplicados | Rechazado |
| Dos eventos, misma sede/fecha/hora | Rechazar segundo; estado intacto | Rechazado; sin programar |
| Dos torneos, mismo participante y hora | Rechazar conflicto global | Rechazado |
| Cotizar 4 de fútbol sala | Q2400.00; no cobrar | Q2400.00; Q0.00 ingreso |
| Completar 4 participantes de baloncesto | Premio Q1000.00; campeón disponible | Q1000.00; campeón |
| Completar carrera 4 participantes | Premio Q600.00; FINALIZADO | Q600.00; FINALIZADO |
| Cargar datos de demostración | Ingresos/premios Q0.00 y eventos en juego programados | Q0.00; todos programados |

## 8.2. Comprobaciones automatizadas por criterio

Se ejecutaron **115 aserciones independientes**. La tabla siguiente resume familias de pruebas y sus valores o efectos verificables. El registro completo, con el nombre de cada aserción, está en **docs/pruebas_ejecutadas.txt** dentro del proyecto Java.

| Familia de pruebas | Entradas y criterio comprobado | Resultado obtenido |
|---|---|---|
| Cotizaciones y cobros | 4 inscritos por disciplina: Q2,400; Q3,000; Q2,000; Q1,800; Q1,200. Cotizar no cobra. | Valores exactos y cobros consistentes |
| Premios | Cuatro participantes: fútbol Q1,200; penales Q1,500; baloncesto Q1,000; voleibol Q900; relevos Q600. | Premios calculados al finalizar |
| Resultados | Fútbol 4-1 = 3/0 y ±3; penales 1-1 (4-3) = 2/1; baloncesto 70-60 = 2/1; voleibol 3-2 = 2/1. | Puntos y diferencias correctos |
| Relevos | 50, 51, 52 segundos y una DQ: 10, 8, 6, 0; diferencia cero. | Asignación y descalificación correctas |
| Entradas inválidas | Empate baloncesto, penales iguales, marcador negativo, tiempos repetidos, hora 24 y sede vacía. | Operaciones rechazadas; estado conservado |
| Programación | Coincidencia de sede o participante entre torneos en fecha/hora; reprogramación antes del resultado. | Conflictos rechazados; reprogramación válida |
| Ordenamientos | Desempates por diferencia, victorias y nombre; calendarios cronológicos, ranking de empresas. | Orden verificado por comparadores |
| Invariantes | Duplicación de código/inscrito, tercera inscripción de misma empresa, cupo insuficiente, cotización. | Sin cambios en ingresos ni estado al rechazar |
| Datos precargados | 8 participantes, cinco disciplinas, torneos en juego programados y resultados de demostración. | Ingresos Q0.00, premios Q0.00 |

**Salida de las pruebas:** RESULTADO: 115 correctas; 0 fallidas. El código de cada caso y su aserción pueden comprobarse en src/arenagt/Pruebas.java.

## 8.3. Pruebas de regresión ampliadas

Se añadieron pruebas de despacho polimórfico sin instanceof, cotizaciones concretas de las cinco cuotas, recuperaciones tras resultados incompatibles, relevos con descalificados y casos específicos sobre duplicidad de códigos, cotizaciones fuera de rango, ruta única de cobro, suma de reportes por disciplina, programación con hora inválida, sede vacía, marcadores negativos, tiempos NaN, tiempos cero, participantes incompletos en relevos, orden alfabético, diferencias y victorias acumuladas y orden cronológico en todos los calendarios. La precarga de demostración se verificó con todos sus eventos en juego programados y sin cobros ni premios iniciales.

## 8.4. Evidencia de consola

ArenaGT - gestión deportiva (datos precargados; sin ingresos iniciales)

1 Participante | 2 Torneo | 3 Cotización | 4 Inscripción | 5 Cerrar | 6 Eventos | 7 Programar/reprogramar | 8 Resultado | 9 Tablas | 10 Calendarios | 11 Reportes | 12 Consultar participantes | 0 Salir

Opción: Ingresos: Q0.00 | Premios: Q0.00

Fútbol sala: Q0.00

Fútbol sala con penales: Q0.00

Baloncesto: Q0.00

Voleibol: Q0.00

Relevos: Q0.00

Ranking de empresas:

Alfa | 13

Beta | 10

Gamma | 3

Delta | 0

1 Participante | 2 Torneo | 3 Cotización | 4 Inscripción | 5 Cerrar | 6 Eventos | 7 Programar/reprogramar | 8 Resultado | 9 Tablas | 10 Calendarios | 11 Reportes | 12 Consultar participantes | 0 Salir

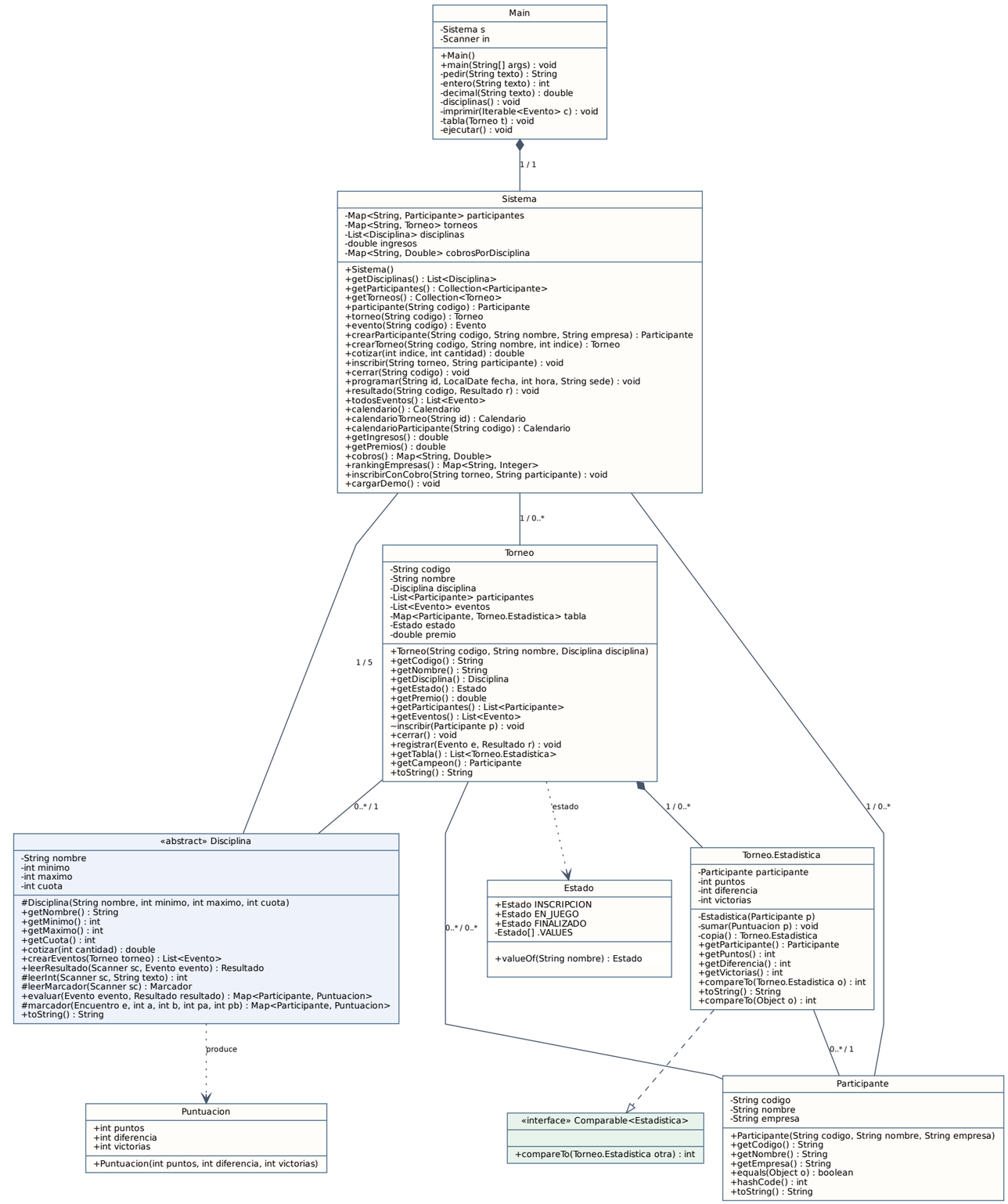
## 8.5. Catálogo complementario de atributos y métodos

Los diagramas A3 anexos contienen las firmas de los métodos. Las siguientes tablas resumen los atributos, visibilidades, tipos y operaciones complementarias existentes en el código fuente; la implementación es la fuente de verdad en caso de detalles de firma.

| Clase | Atributos privados / protegidos | Métodos y resultados públicos principales |
|---|---|---|
| Sistema | participantes: Map; torneos: Map; disciplinas: List; ingresos: double; cobrosPorDisciplina: Map | participante(String): Participante; torneo(String): Torneo; evento(String): Evento; crearParticipante(String,String,String): Participante; crearTorneo(String,String,int): Torneo; cotizar(int,int): double; inscribir(String,String): void; cerrar(String): void; programar(String,LocalDate,int,String): void; resultado(String,Resultado): void; calendario(): Calendario; calendarioTorneo(String): Calendario; calendarioParticipante(String): Calendario; getIngresos(): double; getPremios(): double; cobros(): Map; rankingEmpresas(): Map; cargarDemo(): void |
| Torneo | codigo: String; nombre: String; disciplina: Disciplina; participantes: List; eventos: List; tabla: Map; estado: Estado; premio: double | Torneo(String,String,Disciplina); cerrar(): void; registrar(Evento,Resultado): void; getParticipantes(): List; getEventos(): List; getTabla(): List; getCampeon(): Participante; getPremio(): double; inscribir(Participante): void (paquete) |
| Evento | codigo: String; torneo: Torneo; fecha: LocalDate; hora: Integer; sede: String; finalizado: boolean | Evento(Torneo) [protegido]; getCodigo(): String; getTorneo(): Torneo; getFecha(): LocalDate; getHora(): Integer; getSede(): String; getEstado(): String; isProgramado(): boolean; isFinalizado(): boolean; getParticipantes(): List [abstracto]; compareTo(Evento): int; programar(...): void (paquete); finalizar(): void (paquete) |
| Disciplina | nombre: String; minimo: int; maximo: int; cuota: int | Disciplina(String,int,int,int) [protegido]; getNombre(): String; getMinimo(): int; getMaximo(): int; getCuota(): int; cotizar(int): double; crearEventos(Torneo): List [abstracto]; evaluar(Evento,Resultado): Map [abstracto]; leerResultado(Scanner,Evento): Resultado [abstracto] |

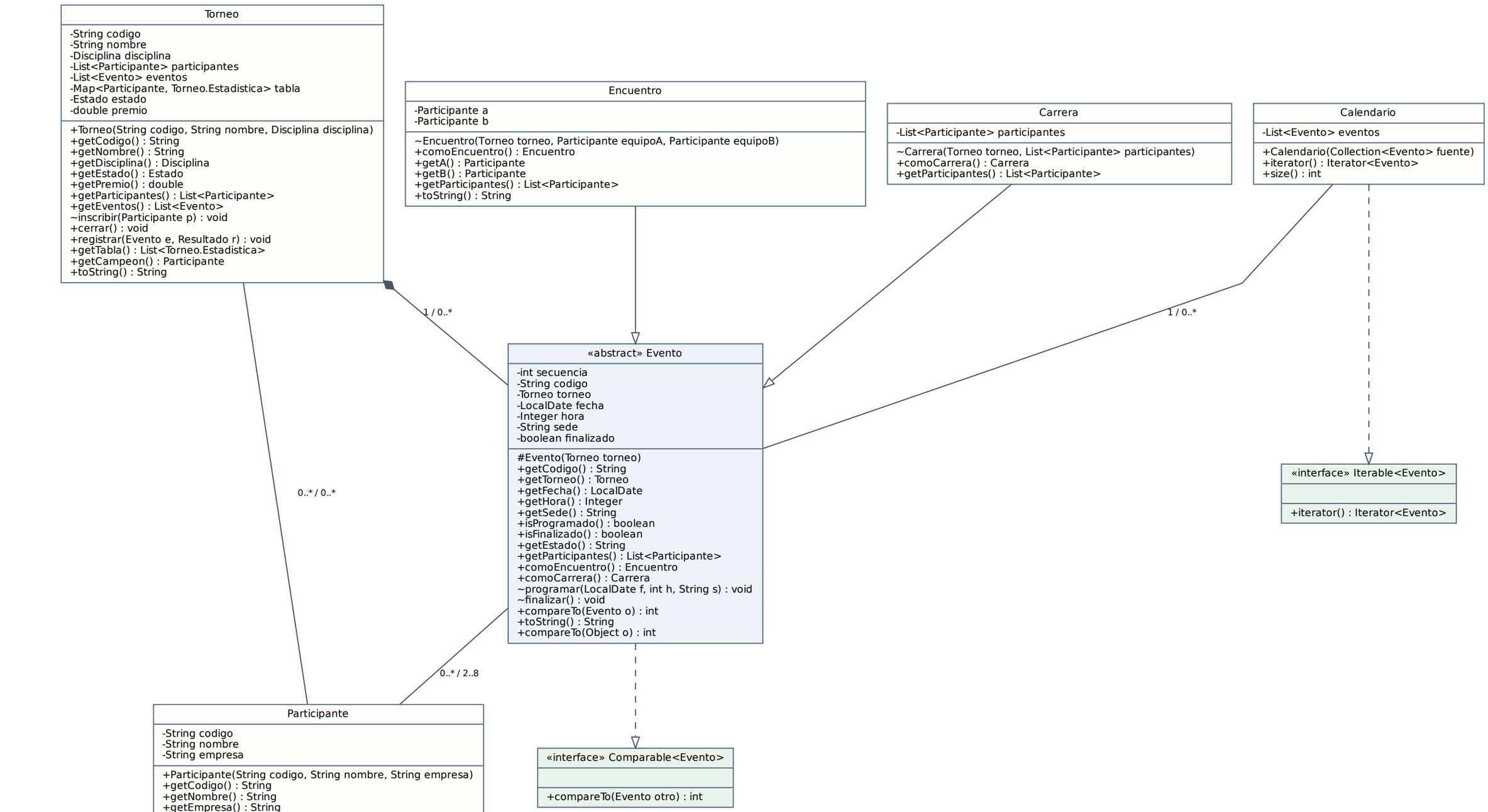
| Componente | Propiedad y responsabilidad / contrato |
|---|---|
| Participante | codigo, nombre, empresa: String privados y finales; constructor valida textos; getters; equals(Object), hashCode() por código. |
| Encuentro y Carrera | Constructor invoca super(torneo); Encuentro conserva a y b; Carrera conserva lista inmutable; ambos sobrescriben getParticipantes(). |
| Calendario | eventos: List privado; constructor filtra y ordena; implementa Iterable; iterator() entrega recorrido inmodificable; size(): int. |
| Torneo.Estadistica | participante: Participante; puntos, diferencia, victorias: int; copia() y sumar(Puntuacion) privados; getters; compareTo(Estadistica) con cuatro criterios. |
| FutbolSala / FutbolPenales | Constructores coordinados con super; evaluar(...) y leerResultado(...) sobrescritos; penales redefine reglas sin duplicar las compartidas. |
| Baloncesto / Voleibol / Relevos | Cada clase implementa evaluar(...) y leerResultado(...); Relevos crea la carrera única mediante crearEventos(...). |
| Resultado y variantes | Resultado es interfaz; Marcador(a,b), ConPenales(a,b,pa,pb), RelevoResultado(tiempos) implementan contrato. |
| Puntuacion / Main / Util | Puntuacion almacena puntos, diferencia, victorias; Main contiene main(String[]) estático, menú y lectura; Util centraliza validaciones y formato monetario. |

# Uml Gestion



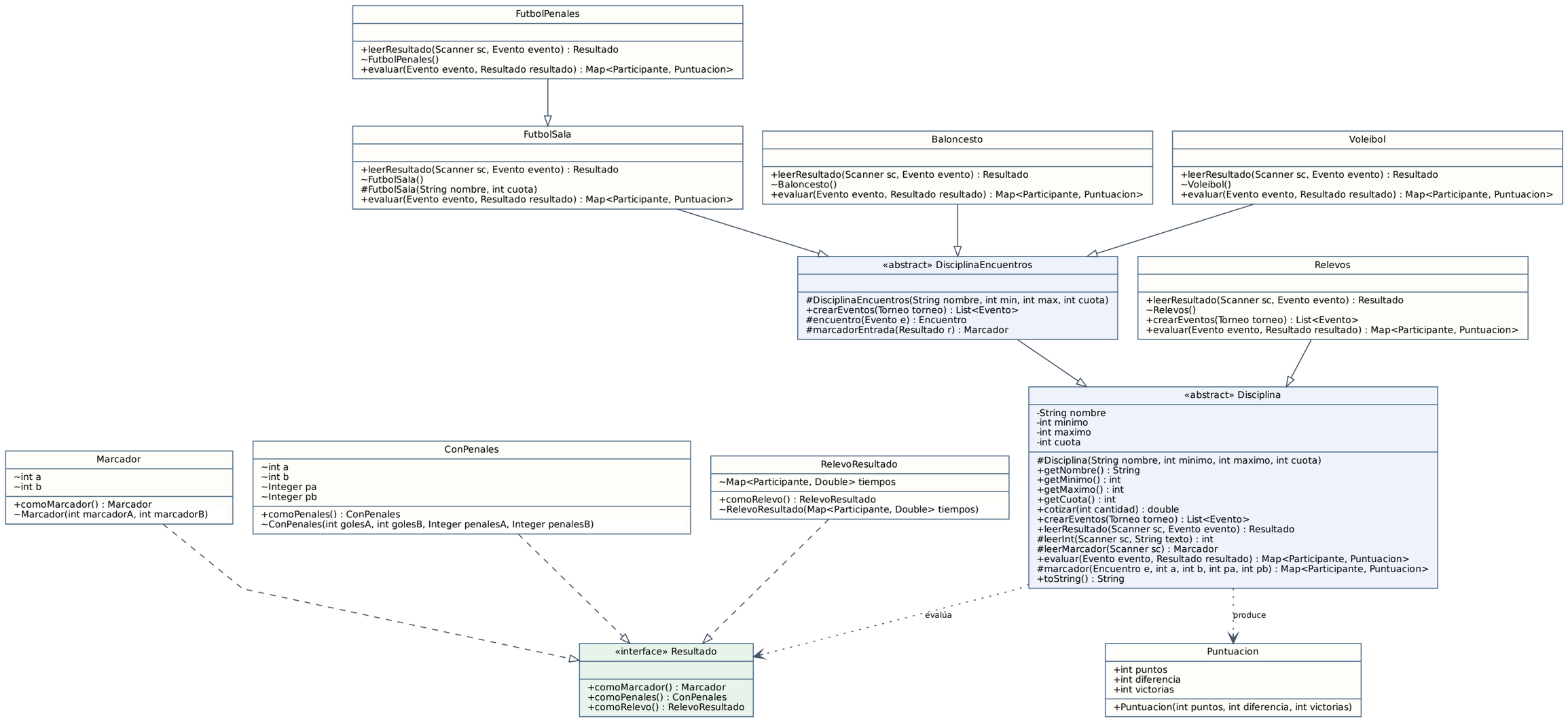

UML: + publico  # protegido  - privado  ~ paquete | triangulo=herencia | linea discontinua=interfaz | rombo=composicion

# Uml Eventos Calendarios




UML: + publico  # protegido  - privado  ~ paquete | triangulo=herencia | linea discontinua=interfaz | rombo=composicion

# Uml Disciplinas




UML: + publico  # protegido  - privado  ~ paquete | triangulo=herencia | linea discontinua=interfaz | rombo=composicion